KEEL

A documentation and working standard for AI-developed projects.

Version 2.0.0 (October 31, 2026) - Rick Flashman. Formerly the AI Project Documentation Standard (1.0.0 to 1.5.1).

Overview

This document defines the baseline documentation files, folder structure, and working rules for any AI-developed project. Like the keel of a ship, it keeps a project steady and on course when the AI tool, the AI model, or the developer changes. It keeps the AI's work organized, supportable, and token-efficient.

This document has two uses:

- **Guide for developers:** It explains how to create and organize a new project.
- **Setup prompt for AI coding assistants:** Claude Code, GitHub Copilot, or another assistant that reads `AGENTS.md` (such as Codex or Cursor) can follow it to set up a new project, or to bring an existing project's documentation up to this standard.
- **Source of the KEEL skill:** the same standard is also installed once, as a set of skills, so it works in every project without this file (see *The KEEL skill*).

> **Important**
>
> **How to use it:** Put this file in the project's root folder and tell your AI coding agent: "*Read KEEL.md and do what it says.*" It works for new and existing projects. The AI asks for your approval before it changes any existing file.

> **Note**
>
> **Terms:** *Required* items (numbered 1-8 in the table) are created at initialization, in numbered order. *Recommended* items are created when first needed. *Optional* items are created only if needed, but `DESIGN.md` and `DESIGN-GUIDE.html` are required if the project has a user interface (UI). *Teams only* items are used when more than 1 developer works on the project.
>
> **Scope:** Version control (branching, merging, pull requests) is not covered. When the project uses branches, the work board in `PLAN.md` records the branch name, nothing more.

Workflow

1. A developer puts `KEEL.md` in the project's root folder.
2. The AI reads it, checks the project (see Check first), and then sets up, migrates, or upgrades the documentation.
3. Afterward, the developer may keep, move, archive, or delete the file. While it is in the root folder, it is the authority (see below). If it is absent, the AI follows the project's existing docs and the developer's instructions.

The KEEL skill

The standard is also packaged as 6 skills, generated from this file by `build/build.py` in the repository, with the same version. A developer installs them once at user level, and they work in every project without this file. Each skill is 1 task; `keel` is the general one for a developer who is not sure which task they need, or who wants to bring a project to the newest version.

Skill	Task
`keel`	Looks at the project, chooses the path (new setup, migration, upgrade, or switch), and does it
`keel-init`	New setup from the templates, for a project with no docs yet
`keel-upgrade`	Migrate existing docs, upgrade an older version, or switch a `KEEL.md` project to the skill
`keel-check`	Write or run the docs check script, and write a drift report
`keel-as-built`	The as-built review

`keel-interview`	The SPEC interview

- **Claude Code:** the repository is a plugin marketplace. Run `/plugin marketplace add DominusFL/keel`, then `/plugin install keel@keel`. The skills are `/keel:keel`, `/keel:keel-init`, and so on. Updates come from the repository through `/plugin`.
- **Codex, Cursor, GitHub Copilot, or Claude Code without the plugin:** run `npx skills add DominusFL/keel -g` (needs Node), or run `install.sh` from the repository, which needs only `git`. Both install the same skill folders into each tool's user-level skills folder, and both can update them later.

The `README.md` of the repository has the full install and update steps. A project that uses this file can switch to the skill at any time (see *Switch to the skill* under Initialization rules), and a project set up by the skill can add this file later; the one in the root folder is then the authority.

Authority and file handling

When `KEEL.md` is in the project root, it is the authoritative reference for project structure, documentation standards, naming conventions, required, recommended, and optional artifacts, and templates and seed content, including for documentation created later. Direct developer instructions and explicit project-specific requirements override it. When project docs disagree with each other, the Precedence list in `AGENTS.md` says which one wins.

AI assistants may read and use the file, but must not delete, rename, move, modify, archive, or otherwise manage it without direct, explicit developer instruction. The developer controls its location and lifecycle, and may ask an AI assistant to update it.

When `KEEL.md` is absent and the KEEL skill is installed, the skill is the reference for the same things. When both are present, `KEEL.md` wins; if their versions differ, tell the developer before you continue.

Initialization rules

- **Check first:** Before you create anything, check the project and choose 1 path:
  - **Upgrade:** `AGENTS.md` (or `CLAUDE.md` before 1.2.0) has a version stamp older than this document's version, and there is no other documentation to bring in. A stamp that names *AI-PROJECT-DOCS-STANDARD* is an older version of this document (KEEL 1.x). Follow the upgrade-only path in *Migrating or upgrading an existing project* (near the end of this document).
  - **Migration:** the project has any file that the Inventory step would find, even a single `README.md` or rules file. Follow *Migrating or upgrading an existing project*.
  - **New setup:** the project has none of these. Create the files from the templates.
  - **Switch to the skill:** the KEEL skill is running, `KEEL.md` is in the root folder, and the developer asked to switch. Run the Upgrade path first if the stamp is older than the skill's version. Then set the stamp to the skill's version and, only on the developer's explicit instruction, delete `KEEL.md` or move it where the developer says. Report what changed.

  If the root folder also holds `AI-PROJECT-DOCS-STANDARD.md`, it is an older version of this document: follow this file, and tell the developer. When the upgrade is complete, ask the developer once whether to delete the old file; do not delete, move, or archive it without that answer. Tell the developer which path you chose and why before you continue. When the path is complete, offer the as-built review if the project has code.

- **As-built review:** If the project has code, offer this review after the setup, migration, or upgrade is complete and reported. The developer may decline or choose only some areas. On approval, follow these steps in order:
    1. **Feature inventory.** List the features the code implements, each with its entry points (routes, commands, screens, jobs, public modules) and folders. Show the list. The developer confirms, renames, merges, or drops items. Do not read deeper until the list is confirmed.
    2. **Area map.** Fill the area map in `ARCHITECTURE.md` for the confirmed areas: code, entry points, tests, command, dependencies (from imports). Set Confidence to Inferred on every row, and mark the table "Inferred from code, needs developer review".
    3. **One area at a time, with a budget.** Read 1 area, write or update its `ARCHITECTURE.md` content (stack, folders, commands, data model, operations), then stop and report before the next. Use a subagent per area where the tool allows, so the main context stays small. If an area exceeds the budget, set its Confidence to Low and record where reading stopped.
    4. **SPEC from behavior.** For each area, write feature text and acceptance criteria that describe what the code does now. If `SPEC.md` is empty or holds only the template, seed it and mark the content "Inferred from code, needs developer review". If `SPEC.md` has content, write the differences as PROPOSED blocks (see *Proposed changes*). The same applies to `DESIGN.md`. Never change accepted SPEC or DESIGN text from code.
    5. **Differences and health.** Report what is built but not specified, what is specified but not built, superseded approaches still in the code, areas with no tests, secrets found, and dependencies with known issues. Lasting findings go to `RESEARCH.md`; questions go to `HANDOFF.md` under Waiting. Set the Verified date in `ARCHITECTURE.md`, and in `SPEC.md` once the developer has reviewed it.
    6. **Stop conditions.** Change no code. Do not run tests unless the developer asks.
- **Existing files:** Never overwrite a file or folder from this document that already exists. Merge the new content in, or propose the change, and ask the developer first.
- **Missing information:** If you need details the developer has not provided, such as the ticket ID or the project description, ask once. If they still do not supply them, leave the placeholder.
- **SPEC interview:** After you seed or migrate `SPEC.md`, offer to interview the developer about its gaps: edge cases, non-goals, trade-offs, constraints, glossary terms, and acceptance criteria. Ask a few questions at a time, and skip questions the docs already answer. Write the resulting SPEC changes as PROPOSED blocks for acceptance (rule 1). The developer may decline. If you offer an as-built review, offer the interview after it.

Folder layout

```
project/
├── KEEL.md                (this file, or the KEEL skill in its place; retained at the developer's discretion)
├── README.md
├── AGENTS.md
├── CLAUDE.md
├── .github/
│   └── copilot-instructions.md
├── .claude/               (optional)
│   └── skills/
│       └── <skill-name>/
│           └── SKILL.md
├── .env.example           (optional)
├── test-results/          (written by test runs; latest.md; excluded from version control)
└── docs/
    ├── SPEC.md
    ├── ARCHITECTURE.md
    ├── PLAN.md
    ├── RELEASE-CHECKLIST.md
    ├── decisions/             (recommended)
    ├── HANDOFF.md             (recommended; created when first needed)
    ├── handoff/               (teams only; one <developer>.md per developer, in place of HANDOFF.md)
    ├── plans/                 (optional; one M<n>.md per milestone when the work board grows)
    ├── CHANGELOG.md           (recommended; created at the first release)
    ├── DESIGN.md              (optional; required if the project has a UI)
    ├── DESIGN-GUIDE.html      (optional; required if the project has a UI)
    ├── USER-GUIDE.md          (optional; for projects with end users)
    ├── RUNBOOK.md             (optional; split from the ARCHITECTURE.md Operations section)
    ├── images/                (optional; screenshots and images for the docs)
    ├── MOCKUP.md              (optional)
    ├── RESEARCH.md            (optional)
    └── archive/               (optional; migration only)
        └── INDEX.md           (lists each archived file and its status)
```

The docs check script (see *Docs check and drift report*) lives where the project keeps its tests, or in `scripts/` if the project has no test folder.

The documentation files

#	File	Status	Purpose	Seeded from the ticket
1	`README.md`	Required	For humans: what the project is, how to run it, owners and support, links	Full description + link
2	`docs/SPEC.md`	Required	What and why: goals, users, glossary, features, user stories, constraints, acceptance criteria, non-goals	Requirements
3	`docs/ARCHITECTURE.md`	Required	How: stack, folders, commands, data model, APIs, auth, hosting, deployment, operations, testing, the area map, diagrams	Approach, deployment notes
4	`docs/PLAN.md`	Required	Execution: milestones, the work board, Definition of ready, Definition of done	Rarely
5	`AGENTS.md`	Required	AI working rules, precedence, and lists of the docs and skills, shared by all AI coding tools	n/a
6	`CLAUDE.md`	Required	Loads `AGENTS.md` into Claude Code with an import line, plus optional Claude-only notes	n/a
7	`.github/copilot-instructions.md`	Required	Points GitHub Copilot to `AGENTS.md` and its listed skills, plus optional Copilot-only notes	n/a
8	`docs/RELEASE-CHECKLIST.md`	Required	Release gate: one check per SPEC acceptance criterion, the non-test release steps, and the latest release result	n/a
-	`docs/decisions/`	Recommended	One short note per major decision, numbered in order. Create the folder with the first decision.	n/a
-	`docs/HANDOFF.md`	Recommended	Short handoff to the next session: next action, open items, traps. About 40 lines, dated bullets. Create it at the first session close, or earlier to record something important.	n/a
-	`docs/handoff/<developer>.md`	Teams only	One handoff per developer, same template and rules as `HANDOFF.md`. Used in place of `HANDOFF.md` when more than 1 developer works on the project.	n/a
-	`docs/CHANGELOG.md`	Recommended	Release history, newest first: what was added, changed, and fixed in each release. Create it at the first release.	n/a
-	`docs/plans/M<n>.md`	Optional	One work board per milestone, used when the board in `PLAN.md` passes about 15 open rows	n/a
-	`docs/DESIGN.md`	Optional (required if the project has a UI)	UI/UX: colors, type, spacing, components, layout, accessibility, wireframe links	UI/UX or design notes
-	`docs/DESIGN-GUIDE.html`	Optional (required if the project has a	Human-readable, visual version of `DESIGN.md`, so graphic designers have an	n/a

		UI)	easy reference. Regenerated from `DESIGN.md` every time `DESIGN.md` changes; never edited by hand.	
-	`docs/USER-GUIDE.md`	Optional (for projects with end users)	Task-based guide for the people who use the product but do not develop or maintain it	n/a
-	`docs/RUNBOOK.md`	Optional	Operations guide for support staff: services, logs, common failures and fixes, backups, contacts. Split from the `ARCHITECTURE.md` Operations section when it passes a page.	n/a
-	`docs/images/`	Optional	Screenshots and images used by the docs, such as the user guide	n/a
-	`docs/MOCKUP.md`	Optional	Mockup-only instructions, temporary sample data, and project-specific mockup decisions	n/a
-	`docs/RESEARCH.md`	Optional	Results of research and comparisons, including options not chosen, kept for later sessions	n/a
-	`docs/archive/`	Optional	Original documentation files moved by a migration, kept for history only, with `INDEX.md` giving each file's status. Never delete them, and never use them as a source of current information.	n/a
-	`.claude/skills/<skill-name>/SKILL.md`	Optional	Project-specific skills, one folder per skill	n/a
-	`.env.example`	Optional	Names of the required environment variables, with no real values. Required if the project uses environment variables.	n/a

Sections that do not apply: Create every required file and keep all its headings. If a section does not apply, say why in the file. Example: the `ARCHITECTURE.md` Auth & security section says the project has no authentication.

Files a developer removes: A developer may decide not to use a file from this document, even a required one. That decision overrides this document. List each such file on the "Not used" line of the `AGENTS.md` Docs list. Never recreate a file on that line, and never report it as missing. If a file from this document is missing and is not on that line, ask the developer once whether to recreate it, and record the answer there.

Projects with no UI: Do not create `DESIGN.md` or `DESIGN-GUIDE.html`, and leave them out of the `AGENTS.md` Docs list. If the project later gains a UI, create both from the templates.

Projects with end users: Create `USER-GUIDE.md` once end users have a working feature to use. Write it in ASD-STE100 Simplified Technical English (STE): short sentences, active voice, and approved words. Describe current behavior as tasks, and put screenshots in `docs/images/`.

Projects with more than 1 developer: Count developers, not tools: a solo developer who uses 2 AI tools is still 1 developer. The work board in `PLAN.md` holds the shared state: who holds which item, which files it touches, and its status. Each developer has a handoff file `docs/handoff/<developer>.md` (the developer's name in lowercase with no spaces, such as `rick.md`), rewritten only by that developer's sessions, and `docs/HANDOFF.md` is not used. The Handoff file line in `AGENTS.md` names the form the project uses. A board Owner is a developer name, with the tool after it when useful ("Rick / Claude Code").

Create other optional files only when needed, from their templates, and add them to the `AGENTS.md` Docs list.

Research versus decisions: Use `docs/RESEARCH.md` for research results or data that do not belong in the plan, specs, architecture, or decisions.

- **Decision note:** records *what* we chose and *why*, in a sentence or two. Example: "We chose A over B because it was faster." The AI must not reverse it.

- **Research entry:** records the *evidence* behind a choice, such as the actual test results for A and B. If A later fails, the entry shows whether B is a viable fallback. Research entries may be updated.

When a decision rests on research, the decision note links to the research entry.

Acceptance criteria versus the release checklist: `SPEC.md` defines correct behavior. Each acceptance criterion has an ID (`AC-1`, `AC-2`, and so on) and its test cases, grouped by feature or milestone, so development can test one part at a time. `RELEASE-CHECKLIST.md` proves a release meets all of them: one check per ID, plus non-test release steps such as updating the version and the docs (rule 6).

Proposed changes to SPEC and DESIGN: The AI never edits accepted text in `SPEC.md` or `DESIGN.md` (rule 1). It writes each proposal in the file, next to the text it changes, as a PROPOSED block, never only in chat:

```
> PROPOSED 2026-10-31 (Claude Code): change AC-3 to allow an empty email.
> Reason: the form now has a guest path. Code: src/forms/signup.js.
```

A PROPOSED block is not a requirement and not a design. The developer accepts it by applying the edit and deleting the marker, rejects it by deleting the block, or tells the AI to do either. A block is named by its file and the AC ID or heading it changes (`SPEC.md AC-3`). The AI lists pending blocks in its task summary and, at the next close, in `HANDOFF.md` under Waiting. No block may remain at a release (rule 6).

Session handoff versus lasting records: `docs/HANDOFF.md` is a short, temporary handoff between sessions, tools, or developers, not a general record of what the AI remembers. Before writing to it, the AI routes each item to its proper home and prints the routing (rule 5):

Content	Goes in
A choice and why it was made	`docs/decisions/`
Test results, options compared, fallbacks	`docs/RESEARCH.md`
Milestones and milestone-level next work	`docs/PLAN.md`
Commands, structure, stack, lasting traps	`docs/ARCHITECTURE.md`
Release check results	`docs/RELEASE-CHECKLIST.md`
Requirement or UI changes	PROPOSED block in `SPEC.md` or `DESIGN.md`; listed in `HANDOFF.md` under Waiting until accepted

Only leftovers go in `HANDOFF.md`: the immediate next action, questions waiting on a person, unconfirmed assumptions, decisions made without asking, and traps found this session that have not yet moved. It never overrides the other docs. The same routing table sits at the top of the `HANDOFF.md` template, so the AI sees it each time it edits the file.

AI instruction files: `AGENTS.md` is a shared format that many AI coding tools read, including Claude Code and GitHub Copilot. It holds all the AI working rules, so a project keeps working as tools change. The other two files only point to it, plus optional notes for their own tool. Do not copy rules from `AGENTS.md` into either one.

- `CLAUDE.md` starts with the line `@AGENTS.md`, which imports `AGENTS.md` into Claude Code. The import works in every current Claude Code setup, including sessions that cannot read `AGENTS.md` directly, and never loads the rules twice. Do not replace it with a symbolic link (links need extra permissions on Windows).
- `.github/copilot-instructions.md` points GitHub Copilot to `AGENTS.md` for setups that do not read `AGENTS.md` directly. Only GitHub Copilot reads it; *Microsoft Copilot* and *Microsoft 365 Copilot* do not. Note the leading dot in `.github`, and use the exact file name. It needs no GitHub repository or version control. Create it even if you only use Claude Code, so others can use GitHub Copilot with the same rules.

Project skills

Skills specific to one project live with that project, not in a developer's personal `~/.claude/skills/` folder, where they would apply to every project on that machine.

- **Location:** `.claude/skills/<skill-name>/SKILL.md`, one folder per skill. Create `.claude/skills/` if it does not exist (note the leading dot).
- **Naming:** Name the file exactly `SKILL.md`. The folder name must match the `name:` line inside it.
- **Using a skill:** Claude Code or GitHub Copilot can pick one up automatically when it is relevant, or a developer can call it by typing `/<skill-name>`.
- **Listing:** When you add, rename, or remove a skill, update the Skills list in `AGENTS.md`. Some tools do not scan `.claude/skills/` on their own, so that list points them there.

Starting point for a new skill:

```
---
name: <skill-name>
description: One sentence on what it does and when to use it.
---

# Skill Name
Instructions for the AI go here.
```

Linking to the project tracker

Most projects have a ticket, work item, or issue in a tracker such as Jira, Azure DevOps, GitHub Issues, or Linear, with a description of the project. Where the table lists a "Seeded from the ticket" section, copy ("seed") the file's first draft from the matching part of that description (for example, a Requirements or Approach section). Which tracker, and what its sections are called, is the project's choice; this document does not depend on any product.

- `README.md` and every Markdown file in `docs/`, except decision notes and files in `docs/archive/`, start with a header line that names the tracker and the ticket: `> Tracker: TICKET-# (name of project)`, with the tracker's name in place of `Tracker` and the ticket ID in place of `TICKET-#`, for example `> Jira: BOOK-12 (Booking Portal)` or `> GitHub: #42 (Booking Portal)`. The README also links to the ticket and starts with the full ticket description. Projects set up under 1.x keep their `> Jira:` headers; they already follow this form.
- Include the ticket ID in the names of work items and in any notes or messages about the work. Decision notes carry no ticket ID.
- If no ticket ID has been assigned yet, leave the `TICKET-#` placeholder. The developer fills it in later, or gives it to the AI to replace every placeholder.
- If the project has no tracker, replace each header with `> Project: (name of project)`, omit the ticket link and ticket IDs, and ask the developer for a project description to use instead.

Risk classes

Every change gets 1 class before work starts. The class is defined here once; rules 6, 9, and 10 and the `AGENTS.md` template cite it instead of repeating it. The AI names the class in the ready card or the task summary, so the developer can correct it.

Class	Trigger	Tests (rule 9)	Review (rule 6)	Plan (rule 10)
Small	1 area, no shared code, no ask-before-touching path	Targeted	None	Direct; a ready card if the change touches several files or its approach is unclear
Shared	Shared code (such as core utilities), the data model, auth, build configuration, a dependency upgrade, a refactor across areas, or a path on the ask-before-touching line in `AGENTS.md`	Affected; ask before a full regression	Fresh-context review of the diff against the acceptance criteria, before the task is done	Ready card, approved first

Release	A release	Full regression	Fresh-context review and drift report	Release checklist

Rules

1. **Humans own SPEC and DESIGN (when it exists),** including the acceptance criteria and their IDs. After the first draft is seeded at setup, the AI may only *propose* edits, as PROPOSED blocks in the file, and must wait for acceptance. Otherwise requirements quietly drift to match whatever the code does.
2. **The AI keeps the docs current.** Every time the application changes, the AI updates the affected files in place, in the same task, not afterward. To find them, it searches `docs/` for each path, name, label, and behavior it changed; the area map and the Glossary give the names. It writes SPEC and DESIGN edits as PROPOSED blocks in the same task, never skips them silently, and never defers doc updates to `HANDOFF.md`. Never create a new version of a doc (such as `SPEC-v2.md`) beside the old one; that leaves several conflicting versions. To change a decision, follow rule 3. How to handle each file:
   - **Update directly:** ARCHITECTURE (including the area map), PLAN (including the work board), the `AGENTS.md` Docs and Skills lists, `.env.example` (when environment variables change), and, when used, MOCKUP, RESEARCH, RUNBOOK, and USER-GUIDE (current user-facing behavior only).
   - **Propose:** SPEC and DESIGN, as PROPOSED blocks (rule 1).
   - **Regenerate:** `DESIGN-GUIDE.html` (when used) every time `DESIGN.md` changes.
   - **`RELEASE-CHECKLIST.md`:** keep one check per accepted acceptance criterion; record results only by running the checks (rule 6).
   - **`HANDOFF.md`** (when used): rewrite at session close through the close routine, or to record something important right away (rule 5), not after every change.
   - **`CHANGELOG.md`** (when used): add one entry at the top at each release (rule 6). Do not rewrite old entries.
3. **Decisions are never rewritten.** To change one, write a new note that supersedes it, then mark the old note's Status as Superseded (the only edit allowed). If 2 notes get the same number, the later merge renumbers its note and fixes the Superseded references.
4. **A task is not done until it meets the Definition of done in PLAN.md.** Test the task as rule 9 says; a task does not need a full regression unless rule 9 calls for one. The result must match the ready card, when there was one; anything outside the card is reported, not done.
5. **The session handoff is not a record.** Run the close routine when the developer says the session is closing, before any compact or clear (for example, `/compact` or `/clear`), or when the session has used most of its context:
   1. **Route.** For each item worth keeping, print 1 line: `item -> target doc`, or `item -> HANDOFF (Next | Waiting | Assumed | Decided | Trap)`. Use the routing table.
   2. **Write** the routed items into their docs (PROPOSED blocks for SPEC and DESIGN).
   3. **Rewrite** `HANDOFF.md` (never append) with only what step 1 routed there, and print the line count.

   A close with no routing report is incomplete. `HANDOFF.md` stays under about 40 lines. Every bullet is dated; an item older than 1 session is moved or deleted, never copied forward, and a lasting fact never stays more than 1 session. After a close at the context limit, tell the developer to start a new session from `HANDOFF.md`; do not continue on a compacted context. If something important would be lost sooner, record it right away in its proper doc or in `HANDOFF.md`, creating the file if needed. On a team, each developer's `docs/handoff/<developer>.md` follows the same rules.

6. **A release is not approved until the release checklist passes.** Before every release, in this order:
    1. A reviewer in a fresh context (a new session or a subagent, not the one that wrote the code) checks the code against the SPEC acceptance criteria, runs each command in `README.md` Run it and `ARCHITECTURE.md` Commands, and reads `README.md` and `USER-GUIDE.md` (if used) against the code. It reports only gaps that affect correctness, the requirements, or the docs. Fix each gap or list it for the developer.
    2. Write the drift report (see *Docs check and drift report*). The developer fixes or accepts each finding.
    3. Run a full regression (rule 9): the whole test suite, the docs check, and every check in `RELEASE-CHECKLIST.md`. Never mark a check passed without running it.
    4. Confirm that no PROPOSED block remains in `SPEC.md` or `DESIGN.md`. Review new or upgraded dependencies (license, maintenance, known vulnerabilities).
    5. Record how each check was verified, replace the Latest release section with the result, set the Verified date in `SPEC.md` and `ARCHITECTURE.md`, and add the release entry to `CHANGELOG.md` (create the file at the first release).
    6. A human gives the final approval.
7. **Never put secrets in code or docs.** If the project uses environment variables, list their names in `.env.example`. Do not paste secrets or customer data into docs or the tracker. If the project uses version control, make sure `.gitignore` excludes `.env` and the `test-results/` folder (rule 9), creating or updating it if needed. This is the only version-control step this document covers.
8. **Keep docs skimmable.** Keep `AGENTS.md` under about 120 lines, because the AI loads it every session; the template is a maximum, so delete the optional lines that do not apply and list only the docs that exist. Keep `CLAUDE.md` and `copilot-instructions.md` to a few lines, and `HANDOFF.md` under about 40. Other docs have no line limit: if one is too long to skim, trim it or split it by topic, leaving a link in the original. `RESEARCH.md` grows by design; split it by topic if it becomes unwieldy. In `AGENTS.md`, for each line, ask whether removing it would cause mistakes; if not, cut it. Do not include what the AI can learn from the code or already does by default. Mark at most 1 or 2 lines "IMPORTANT"; if many lines are marked, none stands out. If the AI keeps breaking a rule, or asks a question that `AGENTS.md` already answers, flag to the developer that the file may be too long or unclear.
9. **Test progressively.** Run only the tests a change needs, as the risk class says. The area map in `ARCHITECTURE.md` lists each area, its code, its tests, its command, its acceptance criteria, and the areas it depends on. Read the map before you read code, and read only the code of the areas you change. An area that a change does not touch is not tested again; its last passing result still holds.
    - **Targeted (the default):** new and updated tests, plus the existing tests for the changed area, plus the docs check.
    - **Affected:** targeted, plus the tests for each area that uses the changed code. Run them for a Shared change. If a targeted test fails in a way that points outside the changed area, run the affected areas without asking.
    - **Full regression:** the whole test suite, the docs check, and every release check. Run it before every release (rule 6) and when the developer asks. For any other reason, ask the developer first and explain why; a Shared change with a wide reach is the typical reason.
    - **Scripted first:** run the saved test commands from the area map. Each run writes a short results file, `test-results/latest.md`: what ran, the pass and fail counts, and details for failures only. The docs check writes `test-results/docs.md` instead, so both results survive the same task. Attach screenshots only for failures or visual checks. Check the exit code and the counts, not only a "passed" line. Test interactively only to debug a failure, to explore a new check before you script it, or for visual checks.
    - **Saved scripts only:** use the project's test framework when it has one. Save each new test script in the project and add its command to the area map. Each test names the acceptance criterion it covers (`AC-n`) in its name or a comment. Do not write throwaway test scripts.
10. **Plan before code.** Classify the change (Risk classes). For a Shared change, a change that touches several files, or a change whose approach is unclear, write the ready card from the `PLAN.md` Definition of ready and wait for approval. Approval of the card approves the plan; its PROPOSED blocks still need acceptance, which the developer may give at the same time. A Small change with a clear approach goes ahead directly. Before a bug fix, reproduce the bug with a failing test or captured error output and save the test; no repro, no fix: ask. Never guess a version pin; read the lock file or ask. If the same fix fails twice, stop, report what you tried, and ask.

11. **Every claim carries evidence.** End every task with the task summary from the `PLAN.md` Definition of done, every line present ("none" if empty); the `AGENTS.md` Reporting section points to it. A test passed only if the results file shows it. A check the AI could not run is reported as "Not run: reason", never as passed. A Shared change is not done until its fresh-context review ran and its findings are fixed or listed. Never weaken, skip, or delete a test to make it pass; report the failure and its cause. After 3 test-fix rounds, stop and report. To dismiss a review finding, cite the file and line that shows it is wrong. Before reporting a task done, re-read the Definition of done and the ready card; this is the cheapest defense against a long context.

Docs check and drift report

Docs check is a saved script in the project's own language, with no new runtime or dependency. It is the one script this document asks for. It is listed in the area map as area *Docs* and runs with the targeted tests of every change; a red check is a failing test. The AI writes it at setup or upgrade and keeps the list of checks, with these definitions, in the script's header comment, so the list survives without this file. It writes its results to `test-results/docs.md`, so the area test results in `test-results/latest.md` survive the same task. It checks:

1. `AGENTS.md` is under about 120 lines and has the version stamp.
2. `docs/HANDOFF.md` and each `docs/handoff/*.md` is under about 40 lines, and every bullet starts with a date.
3. Every `AC-n` in `SPEC.md` outside PROPOSED blocks (lines that start with `>`) has a row in `RELEASE-CHECKLIST.md`, and every row has such an AC.
4. Every relative path and link in `README.md` and `docs/` exists. For every command in the area map and in `ARCHITECTURE.md` Commands, the script file or first word it names exists; the check does not run it.
5. Every file in `docs/` is listed in `AGENTS.md` or on its Not used line (contents of `docs/decisions/`, `docs/images/`, `docs/plans/`, `docs/handoff/`, and `docs/archive/` are covered by their folder's entry).
6. Every doc, except decision notes and `docs/archive/`, starts with a header line of the form `> Name: text`.
7. No file in `docs/` has a standard doc name plus a suffix such as `-v2`, `-old`, `-new`, `-copy`, or `-final`.
8. PROPOSED blocks older than 14 days are listed (a warning, not a failure).
9. No assignment-shaped secret in docs: `password`, `secret`, `api_key`, or `token`, followed by `=` or `:` and a value, or a private key header.

Drift report is a read-only report the AI writes before every release (rule 6) and whenever the developer asks ("run a drift report"). It never blocks and changes nothing. Each finding carries a certainty, and findings are never averaged:

Finding	Certainty	How found
A SPEC feature or AC with no code behind it	Evidence	Area map ACs column against the code
Code with no SPEC coverage	Evidence	Folders not in any area map row, or rows with no ACs
An AC with no test, or a test with no AC	Certain	Area map ACs column against tests that cite `AC-n` in a name or comment
A command, folder, file, or link in the docs that does not exist	Certain	The docs check (paths, links, commands); the reviewer for anything else
A doc section older than the code it describes	Inference	The Verified date against the change history: version-control history if any; else the work board Started dates and `CHANGELOG.md`; else "Not run: no change history". Compares dates, not behavior.

The report goes in the task summary. Lasting findings go to `RESEARCH.md`. At a release, each finding is fixed or accepted by the developer.

Starter templates

Copy these into each new project. Replace the placeholders (`Tracker`, `TICKET-#`, name, and web link if available), and follow or delete the guidance in parentheses. The `AGENTS.md` template is a maximum: at setup, delete the optional lines that do not apply (teams, area owners, `docs/plans/`, `RUNBOOK`), and in its Docs list keep only the files that exist in the

project, plus `docs/decisions/`, `docs/RESEARCH.md`, `docs/HANDOFF.md`, and `docs/CHANGELOG.md`, which stay listed even before they exist so the AI knows to create and use them. Keep the "Not used" line, with "(none)" until the developer removes a file.

README.md

```
> Tracker: TICKET-# (name of project)
(Ticket web link)

## Description
(Full ticket description)

## Run it
(Commands from the ticket)
- Install: `...`
- Dev: `...`
- Test: `...`
- More commands added during development: see `docs/ARCHITECTURE.md`

## Owners and support
- Owner: (who owns the project)
- Report issues: (where, such as the tracker project or a contact)

## Docs
See the `docs/` folder. `AGENTS.md` has the current list of documentation files and what each one is for.
```

docs/SPEC.md

```
> Tracker: TICKET-# (name of project)
Verified: YYYY-MM-DD (last developer review against the code; the only date to trust)

## Goals
## Users
## Glossary
(One line per domain term: term, meaning. One term per concept. Use the same term in code names, the docs, and the user guide.)
## Features & user stories
(Group by area; the same areas appear in the area map in ARCHITECTURE.md. Name the files and interfaces involved, where known.)
## Constraints
(Non-functional requirements: performance, compatibility, security, accessibility, localization. Give each a testable acceptance criterion
where possible.)
## Acceptance criteria & test cases
(Group by feature or milestone. Give each criterion an ID and its test cases. Each ID has a matching check in RELEASE-CHECKLIST.md. End
each group with an end-to-end criterion that proves the feature works. The AI proposes changes as PROPOSED blocks next to the text they
change; see AGENTS.md Making changes.)
### (Feature or milestone) - Area: (name)
- **AC-1:** (Testable: "Form shows an inline error for an invalid email.")
  - Given:
  - When:
  - Then:
## Non-goals
(What is out of scope.)
## Open questions / assumptions
```

docs/ARCHITECTURE.md

```
> Tracker: TICKET-# (name of project)
Verified: YYYY-MM-DD (last as-built review or release; the only date to trust)

## Stack
## Folder layout
## Commands
(Build, run, test, and lint commands added during development. Commands from the ticket stay in README.md.)
## Data model & APIs
(Include migrations, personal data, and retention, or say the project has none.)
## Auth & security
## Hosting & deployment
## Operations
(Logging, error handling, backups and retention, monitoring, and what to do when it fails. Split to docs/RUNBOOK.md when this passes a
page.)
### Known traps
(Lasting surprises and workarounds, moved here from HANDOFF.md: what, why, and how to avoid it.)
## Testing
- What must be tested: (e.g., business logic, API endpoints, key user flows)
- Tools: (e.g., the project's test framework; a saved script only if there is no framework)
- Minimum bar: (e.g., new code ships with tests; targeted tests and the docs check pass for each change; a full regression passes before
each release)
- How to run tests: run the saved commands from the area map. Save each new test script in the project and add its command to the map; no
throwaway scripts. Each test names the acceptance criterion it covers (AC-n) in its name or a comment. Test interactively only to debug a
failure, to explore a new check before you script it, or for visual checks.
- Results file: `test-results/latest.md`. Each saved command writes it itself, so every run produces it. Each run overwrites it with what
ran, the pass and fail counts, and details for failures only. Attach screenshots only for failures or visual checks. Check the exit code
and the counts, not only a "passed" line.
- Docs check: (`...`). The list of checks is in the script's header comment. Runs with every change; writes `test-results/docs.md`.
- Full regression command: (`...`)
### Area map
(One row per area. Use the feature groups from SPEC.md as the areas. Read this map before you read code, and read only the areas you
change. Update the map when code, tests, or areas change. Confidence: High = the developer reviewed the row; Inferred = from code, not yet
reviewed; Low = incomplete read, and the row says where reading stopped.)
| Area | Code (folders or files) | Entry points | Tests | Command (this area only) | ACs | Depends on | Confidence |
|---|---|---|---|---|---|---|---|
| (Login) | (src/auth/) | (src/auth/login.js) | (tests/login.test.js) | (`npm test -- login`) | (AC-1, AC-2) | (Data model, Auth) | (High)
|
| Docs | (scripts/check-docs.js) | | | (`node scripts/check-docs.js`) | | | High |
## Links to diagrams
```

docs/PLAN.md

```
> Tracker: TICKET-# (name of project)
(Milestones and the work board. Every Shared or multi-file change gets a board row; a Small one-file change needs none. Track smaller tasks
in the tracker.)

## Definition of ready
A Shared change, a change that touches several files, or a change whose approach is unclear starts only after the developer approves this
card (classes: AGENTS.md Making changes):

    Change: (name, one line)
    Class: (Small / Shared / Release, and why)
    ACs: (added or changed; each written now as a PROPOSED block in SPEC.md or DESIGN.md, named by file and AC ID or heading)
    Files: (paths, with their areas from the area map in ARCHITECTURE.md)
    Tests: (to add or run, and the affected areas)
    Docs: (to update)
    Out of scope: (what this change will not do)

## Definition of done
Before calling a task done:
- Tests added/updated; targeted tests and the docs check pass, plus affected areas if other code uses the changed code (see the AGENTS.md
Testing section; no full regression unless it calls for one)
- Shared change: fresh-context review of the diff against the ACs done; findings fixed or listed
- Area map in ARCHITECTURE.md updated if code, tests, or areas changed
- Task summary in this shape, every line present ("none" if empty):

      Changed: (files)
      Class: (Small / Shared / Release)
      Tested: (command -> pass/fail counts; results file)
      Reviewed: (fresh-context result for a Shared change, or n/a)
      Docs updated: (files) | Docs checked, no change: (files; search terms used)
      Proposed: (PROPOSED blocks by file and AC ID or heading)
      Decided without asking: (one line each)
      Skipped: (step and why)
      Next: (one action)

- Result matches the ready card, if there was one; anything outside it is reported, not done
- Code is readable and documented per the AGENTS.md rules
- Affected docs updated, as listed in the AGENTS.md Docs list (SPEC and DESIGN changes written as PROPOSED blocks; all other docs updated
directly)
- DESIGN-GUIDE.html, if it exists, regenerated if DESIGN.md changed
- USER-GUIDE.md, if it exists, updated if user-facing behavior changed
- RELEASE-CHECKLIST.md updated if acceptance criteria changed
- `.env.example` updated if environment variables changed
- AGENTS.md Docs and Skills lists updated if files or skills changed
- Work board row updated, if the change has one (Status; Done when finished)

## Milestones
- [ ] M1: (name; owner; status)
- [ ] M2:

## Work board
(One row per work item in the current milestone. Claim an item by setting Owner and Status before you start. Two In progress rows may not
list the same file; if another row holds a file you need, do not start: add the dependency and pick another item or ask. Mark Done; delete
nothing. When open rows pass about 15, move this table to docs/plans/M<n>.md and keep only the milestones here. A solo developer keeps the
board too; it stays short.)
| Item | Owner | Areas | Files | Status | Branch or ticket | Started |
|---|---|---|---|---|---|---|
| (Guest signup) | (Rick / Claude Code) | (Signup) | (src/forms/signup.js) | (Ready / In progress / Blocked / Review / Done) | (TICKET-12)
| (YYYY-MM-DD) |
```

AGENTS.md

```
# Project rules
AI working rules for every AI coding tool on this project. CLAUDE.md and .github/copilot-instructions.md only point here. Keep this file under about 120 lines; cut any line whose removal would not cause mistakes.

## Project documentation standard
- Aligned to KEEL 2.0.0 (https://github.com/DominusFL/keel). Update when the docs are realigned.
- If `KEEL.md` is in the project root, read it before creating docs or changing the doc structure; it is the authority. Never delete, rename, move, modify, or archive it without explicit developer instruction. If it is absent, follow the KEEL skill when it is installed, else the existing docs, and keep this section. Developer instructions override both.
- Precedence, highest first: developer instruction > docs/decisions/ > SPEC.md and DESIGN.md (accepted text, not PROPOSED blocks) > ARCHITECTURE.md and PLAN.md > HANDOFF.md and RESEARCH.md > the code (evidence only) > docs/archive/ (never a source).

## Commands
- In README.md (from the ticket) and docs/ARCHITECTURE.md. Record new commands in ARCHITECTURE.md only.

## Conventions
- (Project-specific code style, naming, folder rules)
- Ask before touching: (paths that need a ready card and approval regardless of size, such as auth, data migrations, build config, CI; "(none)" if none)
- Area owners: (optional: area, owner, review required; delete if unused)

## Code
- No clever or compressed code; if performance needs it, comment why.
- Header comment on every file; doc comment on every function/component (what, inputs, outputs); inside, comment only the *why* of non-obvious logic.

## Making changes
- Classify first, then act by class:
  - Small (1 area, no shared code): change directly; targeted tests.
  - Shared (shared code, data model, auth, build config, dependency upgrade, cross-area refactor, or an ask-before-touching path): ready card and approval first; affected tests; then a fresh-context review of the diff against the ACs.
  - Release: docs/RELEASE-CHECKLIST.md.
- Ready card, for a Shared change, a multi-file change, or an unclear approach: the Definition of ready in docs/PLAN.md.
- Bug fix: reproduce first with a failing test or captured error output, and save the test; no repro, no fix. Never guess a version pin. If the same fix fails twice, stop, report, and ask.
- Update the affected docs in the same task, never deferred to the handoff file or a later task. Before you finish, search docs/ for each path, name, label, and behavior you changed (names: area map, SPEC Glossary) and update each hit.
- SPEC and DESIGN: never edit accepted text. Write a PROPOSED block next to the text it changes, never only in chat: `> PROPOSED YYYY-MM-DD (tool): the edit. Reason: ... Code: path.` Name it by file and AC ID or heading. The developer accepts by applying the edit and deleting the marker, or rejects by deleting the block.

## Testing
- Read the Testing section and area map in docs/ARCHITECTURE.md before you read code or run or write a test; read only the areas you change. Each test names the AC it covers.
- Run only what the change needs: new and updated tests, the changed area's tests, the docs check, and the tests of areas that use the changed code. If a failure points outside the changed area, run those areas too. Do not retest unchanged areas.
- Full regression only before a release or when the developer asks; otherwise ask first and say why.
- Never weaken, skip, or delete a test to make it pass; report the failure and its cause. After 3 test-fix rounds, stop and report.

## Reporting
- Lead with the result. No preamble, recap, or effort estimates; lists of at most 5 items. Full content for errors, failing tests, and confirmations.
- Before reporting done, re-read the Definition of done in docs/PLAN.md and your ready card, and end with the task summary it defines, every line present.
- A check you could not run is "Not run: reason", never passed. A pass claim cites test-results/latest.md. To dismiss a review finding, cite the file and line that shows it is wrong.

## Decisions
- Before a structural or technology choice, check docs/decisions/. Never reverse a decision; supersede it: write a new note, then set the old note's Status to "Superseded by NNNN", the only edit allowed.
- Note choices that are hard to reverse or affect multiple files (stack, data model, auth, hosting, major dependencies); skip routine ones. File: docs/decisions/NNNN-short-title.md, next number, no ticket ID, same format as the newest note (first note: `# NNNN: Title`, Status, Context, Decision, Why). On a number collision, the later merge renumbers. Mention new notes in your summary.

## Session handoff
- Handoff file: docs/HANDOFF.md (teams: docs/handoff/<developer>.md, one per developer). At session start, read it if it exists, and the work board in docs/PLAN.md.
- Close routine, when the developer closes the session, before any compact or clear, or when most of the context is used: (1) Route: print one line per item: a choice -> docs/decisions/; evidence or results -> docs/RESEARCH.md; milestones -> docs/PLAN.md; commands, structure, lasting traps -> docs/ARCHITECTURE.md; release results -> docs/RELEASE-CHECKLIST.md; requirement or UI changes -> PROPOSED block; leftovers -> the handoff file (Next | Waiting | Assumed | Decided without asking | Traps). (2) Write the routed items into their docs. (3) Rewrite the handoff file (never append; if missing, create it with Where we left off and those 5 sections) with only what step 1 routed there; print the line count. After a close at the context limit, tell the developer to start a new session from the handoff file; do not continue on a compacted context.
- Handoff file: about 40 lines; every bullet dated; anything older than one session is moved or deleted, never copied forward. Not rewritten after every change; if something important would be lost before the close, record it now in its proper doc or in the handoff file. It never overrides other docs; on a conflict, they win: flag it.

## Always / Never
- Always: mention the ticket ID when describing work; meet the Definition of done before finishing; claim a work board row before a Shared or multi-file change.
- Never: put secrets in code or docs; mark a release check passed without running it; create a new version of a doc (such as SPEC-v2.md) beside the old one; edit docs/archive/ or use it as a source.

## Docs (read the one that matches your task)
- docs/SPEC.md: requirements, glossary, constraints. PROPOSED blocks only.
- docs/ARCHITECTURE.md: area map, structure, commands, operations. Read before structural changes. Update directly.
- docs/PLAN.md: work board, Definition of ready and done. Update directly. (docs/plans/M<n>.md: per-milestone boards, when used.)
- docs/RELEASE-CHECKLIST.md: release gate; follow it before every release. A human approves.
- docs/decisions/: past decisions.
- docs/HANDOFF.md (teams: docs/handoff/<developer>.md): read at session start.
- docs/CHANGELOG.md: entry at each release (create at the first); read only the top entries.
- docs/DESIGN.md: read before touching UI. PROPOSED blocks only. docs/DESIGN-GUIDE.html: generated from it; never edit by hand; regenerate when DESIGN.md changes.
- docs/USER-GUIDE.md: end-user guide in STE; update directly for current user-facing behavior.
```

```
- docs/RUNBOOK.md: operations for support staff. Update directly.
- docs/images/: images used by the docs.
- docs/MOCKUP.md: read before mockups; update directly; never redefines production requirements.
- docs/RESEARCH.md: check before researching again; save research worth keeping (create if needed).
- docs/archive/: migration originals, history only; INDEX.md says what replaced each.
- Not used (developer decision; do not recreate or report missing): (none)

## Skills (.claude/skills/<skill-name>/SKILL.md)
- When a task matches a skill listed here, read and follow its SKILL.md (some tools do not scan this folder).
- (`<skill-name>`: one-line purpose. Leave empty if none.)
```

CLAUDE.md

Keep the `@AGENTS.md` line first and on its own line. Do not copy rules from `AGENTS.md` into this file. A session-start hook that prints `docs/HANDOFF.md` is allowed as a Claude-only convenience; the rules do not depend on it.

```
@AGENTS.md

## Claude-only notes
- When compacting, keep the list of changed files, the test commands that ran, the ready card, pending PROPOSED blocks, and open questions.
(Add only rules that apply to Claude Code and no other tool. Shared rules belong in AGENTS.md.)
```

.github/copilot-instructions.md

Keep the pointer lines first. Do not copy rules from `AGENTS.md` into this file.

```
Follow all rules in AGENTS.md in the project root, including the Docs and Skills lists.
When a task matches a skill listed in AGENTS.md, follow the instructions in
.claude/skills/<skill-name>/SKILL.md.

## Copilot-only notes
(Optional. Add only rules that apply to GitHub Copilot and no other tool. Shared rules belong in AGENTS.md. Delete this section if unused.)
```

docs/decisions/0001-short-title.md *(recommended; create when the first decision is recorded)*

```
# 0001: Short title
Status: Accepted (or Superseded by 000X)
Context: What problem or choice we faced.
Decision: What we chose.
Why: Main reasons, and what we rejected. (Link to the matching docs/RESEARCH.md entry if detailed results exist.)
```

docs/RELEASE-CHECKLIST.md

The release gate. Create it at setup with one check per acceptance criterion in `SPEC.md`, and keep it in step as the criteria change. Before each release, have a fresh-context review against the acceptance criteria and the docs, write the drift report, run a full regression, including every check, and replace the Latest release section. Example prompt: "*Prepare release 1.0: run the release checklist and record the results.*"

```
> Tracker: TICKET-# (name of project)

# Release checklist
Never mark a check passed without running it. Record how each one was verified. A human gives the final approval.

## Requirement checks
(One line per acceptance criterion in SPEC.md. Method: automated test, manual check, or both. For automated checks, the results file from
the full regression is the evidence.)
| ID | Check | Method | Result |
|---|---|---|---|
| AC-1 | (Short description from SPEC.md) | (Automated / Manual) | (Pass / Fail / Not run: reason) |

## Release steps
- [ ] Fresh-context review done: code against the SPEC acceptance criteria, each command in README Run it and ARCHITECTURE Commands run,
README and USER-GUIDE read against the code; each gap fixed or accepted by the developer
- [ ] Drift report written (SPEC with no code: Evidence; code with no SPEC: Evidence; AC with no test or test with no AC: Certain; broken
path, link, or command: Certain; doc older than its code: Inference); each finding fixed or accepted by the developer
- [ ] Full regression passes: the whole test suite and the docs check, using the full regression command in ARCHITECTURE.md (results in
test-results/latest.md and test-results/docs.md)
- [ ] No PROPOSED blocks remain in SPEC.md or DESIGN.md
- [ ] New or upgraded dependencies reviewed: license, maintenance, known vulnerabilities
- [ ] CHANGELOG.md has an entry for this release (create the file at the first release)
- [ ] Version number updated (if the project uses one)
- [ ] README.md, ARCHITECTURE.md, and USER-GUIDE.md (if used) match the release; Verified dates set in SPEC.md and ARCHITECTURE.md
- [ ] DESIGN-GUIDE.html regenerated if DESIGN.md changed
- [ ] .env.example is current; no secrets or customer data in code or docs
- [ ] Open items in the handoff file(s) resolved or accepted for this release
- [ ] (Project-specific steps, such as deployment or accessibility checks)

## Latest release
(Replaced at each release.)
- Release: (version), YYYY-MM-DD
- Checked by: (tool or developer)
- Result: (All checks passed, or each failed, not-run, or skipped check and why)
- Approved by: (developer name)
```

docs/HANDOFF.md *(recommended; create when first needed)*

A short session handoff so the next session (same tool, a different tool, or a different developer) can pick up without guessing. It is not created at initialization. Create it at the first session close, or earlier if something important must be recorded right away. Every bullet is dated, and the file stays under about 40 lines. Example closing prompt: *"I'm closing this session. Run the close routine: route each item, write it to its doc, then rewrite docs/HANDOFF.md."* On a team, each developer keeps `docs/handoff/<developer>.md` from the same template, and `docs/HANDOFF.md` is not used.

```
> Tracker: TICKET-# (name of project)

# Session handoff
Last updated: YYYY-MM-DD by (tool or developer)

Session handoff only; rewritten at each session close, about 40 lines. If anything here conflicts with another doc, the other doc wins.
Routing: a choice -> docs/decisions/ | results or evidence -> docs/RESEARCH.md | milestones -> docs/PLAN.md | commands, structure, lasting
traps -> docs/ARCHITECTURE.md | release results -> docs/RELEASE-CHECKLIST.md | requirement or UI changes -> PROPOSED block in SPEC.md or
DESIGN.md. Only leftovers stay here, in the 5 sections below (Decided = Decided without asking; Traps = Traps found this session). Date
every bullet; move or delete anything older than one session.

## Where we left off
(Two sentences: what was being worked on and its state, naming any file partly changed. History goes in CHANGELOG.md or PLAN.md, not here.)

## Next
- (YYYY-MM-DD) (The immediate next action within the current task, with the file it touches; 1 to 3 bullets. Milestone-level work goes in
PLAN.md.)

## Waiting
- (YYYY-MM-DD) (A question waiting on a named person, or a pending PROPOSED block: file and AC ID or heading.)

## Assumed
- (YYYY-MM-DD) (An unconfirmed assumption, and what would confirm it.)

## Decided without asking
- (YYYY-MM-DD) (A choice the AI made on its own, with the evidence. Target: docs/decisions/ or a PROPOSED block.)

## Traps found this session
- (YYYY-MM-DD) (A surprise or workaround.) Move to: (ARCHITECTURE Known traps / RESEARCH / decisions)
```

docs/plans/M.md *(optional; create when the work board in `PLAN.md` passes about 15 open rows)*

```
> Tracker: TICKET-# (name of project)

# M<n>: (milestone name)
Owner: (name). Status: (Planned / In progress / Done).

## Work board
(Same rules as the board in PLAN.md: claim before you start; no two In progress rows share a file; mark Done, delete nothing.)
| Item | Owner | Areas | Files | Status | Branch or ticket | Started |
|---|---|---|---|---|---|---|
```

docs/CHANGELOG.md *(recommended; create at the first release)*

The release history, for testers, support staff, and other readers. It is not created at initialization. At each release, add one entry at the top, so the AI reads only the newest entries. Keep each entry short. Do not rewrite old entries.

```
> Tracker: TICKET-# (name of project)

# Changelog
Newest release first. Add one entry at the top for each release. Do not rewrite old entries.

## (Version) - YYYY-MM-DD
- Added: (New features)
- Changed: (Changes to existing behavior)
- Fixed: (Bugs fixed)
```

docs/DESIGN.md *(optional; required if the project has a UI)*

Create this file for every project with a UI. Skip it for a project with no UI. The AI proposes changes as PROPOSED blocks (rule 1).

`docs/DESIGN-GUIDE.html` has no template. It is a human-readable, visual version of `DESIGN.md`, so graphic designers have an easy reference. Regenerate it from `DESIGN.md` every time `DESIGN.md` changes, in the same task. Never edit it by hand. Make it one self-contained HTML file that opens in a browser with no other files or build step. Follow the `DESIGN.md` headings, and show the values visually, such as color swatches, type samples, and spacing examples. Start it with a comment saying it is generated from `DESIGN.md` and must not be edited by hand.

```
> Tracker: TICKET-# (name of project)

## Colors & typography
## Spacing & components
## Layout & responsive behavior
## Accessibility
## Links to wireframes and brand design guidelines
```

docs/USER-GUIDE.md *(optional; for projects with end users)*

Create this file once the product has a working feature that end users will use. End users are people who use the product but do not develop or maintain it. Write it for them, not for developers, in ASD-STE100 Simplified Technical English (STE): short sentences, active voice, one instruction per step, and approved words. Use the terms from the SPEC Glossary. Describe how the product works now, not planned features. Put screenshots in `docs/images/` and link them with relative paths.

```
> Tracker: TICKET-# (name of project)

# (Product name) user guide

## What it does
(One or two short sentences: what the user can do with it.)

## Getting started
(How to open, sign in to, or install it.)

## Tasks
### (Task, such as "Book a stay")
1. (Step)
2. (Step)

![(Description of the screenshot)](images/(file-name).png)

## Troubleshooting
- (Common problem): (What to do)

## Getting help
(Who to contact.)
```

docs/RUNBOOK.md *(optional; create when the `ARCHITECTURE.md` Operations section passes a page)*

For the people who run and support the product. Move the Operations content here and leave a link in `ARCHITECTURE.md`. Update it directly when operations change.

```
> Tracker: TICKET-# (name of project)

# Runbook
## Services and where they run
## Logs and monitoring
## Common failures and fixes
## Backups and restore
## Contacts and escalation
```

docs/MOCKUP.md *(optional)*

Create this file only when the project will include mockups or prototypes. Use it for mockup guidance, temporary sample data, and mockup-specific decisions that do not belong in the production requirements or design documents.

```
> Tracker: TICKET-# (name of project)

# Mockup instructions
This file applies only to mockups and prototypes. It does not override `SPEC.md` or `DESIGN.md`.

## Baseline (example)
- Create each mockup as a single-screen, 16:9 SVG.
- Start with a clean, uncluttered pencil-wireframe style using an off-white background and dark gray strokes.
- Light green may be used for callouts that are not part of the interface.
- Use specified colors when color is important to the design.
- Use realistic temporary content based on the vacation resorts, vacations, timeshare, and hospitality industry.

## Project-specific instructions
Add mockup-specific decisions and user feedback here as the mockups evolve.
(None yet.)

## Temporary sample data
List any invented data or examples that should be used consistently. Do not use real customer data, secrets, or other sensitive
information.
(None yet.)
```

docs/RESEARCH.md *(optional)*

Create this file only when development produces research results or data that do not belong in the plan, specs, architecture, or decisions. Record research worth keeping for a future session, such as options tried but not chosen, and lasting findings from the as-built review or a drift report. Skip trivial lookups. Record the choice itself in `docs/decisions/`, and keep the supporting results here.

```
> Tracker: TICKET-# (name of project)

# Research
Newest entries first. Do not delete old entries. If a finding is no longer true, mark it "Outdated (date)" and add the new finding.

## Short topic title (YYYY-MM-DD)
- **Question:** What we needed to find out.
- **Options tried:** Each option and its results (numbers, behavior, limits).
- **Outcome:** What we used, and the main reason.
- **Fallback notes:** Options not used that could replace it, and what to watch for.
- **Sources:** Links, files, or tests the findings came from.
- **Not verified:** Anything uncertain or untested.
```

docs/archive/INDEX.md *(optional; create during migration)*

Lists every file moved into `docs/archive/` and what replaced it, so no session mistakes an old file for a current one. If the index already exists, add new rows to it.

```
# Archive index
Original files moved here during migration, kept for history only. Do not use them as a source of current information. Never delete them.

Status: Migrated (all content moved to the current docs), Partly migrated (current sections moved; the rest was superseded), or Superseded
(not migrated; replaced by a newer source).

| Original path | Status | Replaced by | Archived |
|---|---|---|---|
| (notes/auth-v2.md) | (Superseded) | (docs/decisions/0003-login-method.md) | (YYYY-MM-DD) |
| (SPEC-old.md) | (Migrated) | (docs/SPEC.md) | (YYYY-MM-DD) |
```

Initialization checklist

After creating the initial files, confirm each item and report any that fail.

- [ ] `README.md`, `docs/SPEC.md`, `docs/ARCHITECTURE.md`, `docs/PLAN.md`, and `docs/RELEASE-CHECKLIST.md` exist, from the starter templates, unless the developer chose not to use one and it is on the `AGENTS.md` "Not used" line.
- [ ] Every SPEC acceptance criterion has an ID and a matching check in `RELEASE-CHECKLIST.md`. `SPEC.md` has the Glossary and Constraints headings.
- [ ] `ARCHITECTURE.md` has the Operations section and the area map with a Docs row. The docs check script exists, is listed in the map, and passes.
- [ ] `PLAN.md` has the Definition of ready, the Definition of done, and the work board.
- [ ] A project with a UI has `docs/DESIGN.md` and `docs/DESIGN-GUIDE.html`. A project without one has neither, and `AGENTS.md` does not list them.
- [ ] `AGENTS.md` is under about 120 lines and has the version stamp, the precedence line, and the Project documentation standard, Making changes, Testing, Reporting, Session handoff, Docs, and Skills sections.
- [ ] `CLAUDE.md` starts with `@AGENTS.md`, and `.github/copilot-instructions.md` (exact name) points to `AGENTS.md`. Neither copies rules from `AGENTS.md`.
- [ ] Every tracker header has the tracker name and `TICKET-# (name of project)` filled in, or keeps the placeholder for the developer.
- [ ] Every required section that does not apply says why (for example, no authentication in `ARCHITECTURE.md`).
- [ ] Optional files exist only where needed and are listed in `AGENTS.md`. `docs/HANDOFF.md` was not created, unless a migration or the as-built review wrote to it, and `docs/CHANGELOG.md` was not created, unless a migration moved release history into it.
- [ ] No PROPOSED blocks exist, unless a migration or the as-built review created them; those are listed in the report and in `HANDOFF.md` under Waiting.
- [ ] No file contains secrets or customer data, and `.gitignore` excludes `.env` and `test-results/` if the project uses version control.
- [ ] After a migration: no Superseded content was migrated, and `docs/archive/INDEX.md` lists every archived file.
- [ ] No existing file was overwritten without the developer's approval, and `KEEL.md` was not deleted, renamed, moved, or modified, except by a switch to the skill on the developer's explicit instruction.

Migrating or upgrading an existing project

Use this section when the project already has documentation: another format, a single notes file, or an older version of this standard. The other Initialization rules (existing files, missing information) still apply. Whatever the prompt, change nothing until the developer approves the plan in step 5. Change only documentation, never code, configuration, or dependencies; the docs check script is the one exception.

Upgrade only: If the project already follows an older version of this standard, do steps 1, 8, and 9. Step 8 says when to run other steps.

1. **Back up.** Confirm with the developer that a backup or version-control commit exists. If not, copy the existing documentation to a folder outside the project and tell the developer where it is.
2. **Inventory.** Scan the whole project, including subfolders, for `.md`, `.txt`, `.rst`, `.adoc`, and similar text files, plus `.docx` and `.pdf` files you can read. Include rules files (`CLAUDE.md`, `AGENTS.md`, `.cursorrules`, `.github/copilot-instructions.md`, `.github/instructions/`), decision records, test plans, QA or release checklists, user manuals or help pages, runbooks, wiki exports, TODO lists, changelogs, and release notes. Skip `docs/archive/` and dependency, build, and version-control folders (such as `node_modules`, `vendor`, `dist`, `bin`, `obj`, and `.git`). List any file you could not read and ask how to handle it.
3. **Resolve currency.** Group the inventory by topic. For each topic with two or more sources, find the current source. Use these signals, strongest first:
    1. A direct statement from the developer.
    2. An explicit marker: "superseded", "deprecated", "obsolete", "old", "replaced by", or a link from a newer file.
    3. Agreement with the current code (evidence only; code never changes SPEC or DESIGN by itself).
    4. Dates or version numbers in the content or the file name (such as `v2`, `final`, or `2025-03`).
    5. Version-control history, if it exists.

    Do not rely on file modification dates alone; copies and sync tools (such as OneDrive or SharePoint) change them. Give each file one status: **Current**, **Partly current** (list the current sections), or **Superseded** (name what replaced it). If the signals disagree or are weak, mark the topic **Unresolved**.
4. **Map.** Give each Current item, and each current section of a Partly current file, a target file and section (for example, "requirements in README → `docs/SPEC.md` Goals"). Superseded content gets no target; it stays in the archive. Flag content with no target, content that conflicts with this standard, and content you would have to infer. When a newer source changed a choice recorded in a Superseded file, propose 2 decision notes from the original wording: the old choice ("Superseded by NNNN") and the new one (Accepted). For evidence of the change, such as failed tests, propose a `docs/RESEARCH.md` entry with the old finding marked "Outdated (date)".
5. **Get approval.** Show the developer the inventory, the currency table below, the mapping, the proposed decision notes and research entries, and the draft archive index (step 7). Ask about every Unresolved topic and every Low-confidence row; do not guess. Wait for approval.

Topic	Sources	Current source	Evidence	Confidence
(Login method)	(`auth.md`, `auth-v2.md`, `new-login.md`)	(`new-login.md`)	(Matches the code; newest date)	(High / Medium / Low)

6. **Migrate**, using the starter templates:
    - Migrate only the content mapped in step 4. Reorganize its original wording; do not rewrite or drop it. Never invent content: if a section has no source, keep the heading and write "(Not documented yet.)" Mark content inferred from code "Inferred, needs developer review" in SPEC and DESIGN.
    - Give each acceptance criterion an ID, and create `RELEASE-CHECKLIST.md` with a matching check for each, carrying in existing test plans and release checklists.
    - Build the `ARCHITECTURE.md` area map from the existing code and tests, as its template says. If no command can run one area alone, write "(Full suite only)" and report it. Do not run the tests during the migration unless the developer asks. Write the docs check script and add its row.
    - Put user manuals or help content in `docs/USER-GUIDE.md`, and their images in `docs/images/`. Offer to convert the guide to Simplified Technical English. Put runbooks and operations notes in the `ARCHITECTURE.md` Operations section, or `docs/RUNBOOK.md` if they pass a page.
    - Copy decision records into `docs/decisions/` word for word, keeping their status (Accepted if none). Add the approved decision notes, using "(Not documented yet.)" for any part with no source, and number all notes together in date order. Link each approved research entry from its decision note.
    - Merge the rules files into `AGENTS.md`. Put project-specific rules under Conventions, Claude-only rules under Claude-only notes in `CLAUDE.md`, and Copilot-only rules under Copilot-only notes in `.github/copilot-instructions.md`. Report conflicts with this standard or between files. Then replace `CLAUDE.md` and `.github/copilot-instructions.md` with their templates, keeping those notes.
    - Move lasting content from handoff, session-note, or TODO files to its proper doc, and leftovers to `docs/HANDOFF.md` as dated bullets, with the routing printed.
    - Move changelogs and release notes to `docs/CHANGELOG.md`, newest first.
    - Update links to moved content. Point links to Superseded files at their replacements, not at the archive. Set the `AGENTS.md` version stamp to this document's version.
7. **Archive.** Move the originals into `docs/archive/` under their original paths, so files with the same name do not collide (`notes/auth.md` becomes `docs/archive/notes/auth.md`). For files that keep their path, such as `README.md` and the rules files, save a copy there before editing. Create `docs/archive/INDEX.md` from its template, or add to it, with one row per archived file. Never delete the originals or the backup.

8. **Upgrade.** The version stamp is in `AGENTS.md` (in `CLAUDE.md` before 1.2.0). If it is older than this document's version, apply only the changes since that version, in order, using the Changelog and the blocks below. Show the developer the planned changes and wait for approval, as in step 5. Then apply them and update the stamp.
    - **1.1.0 or earlier to 1.2.0:** Handle the rules files as in steps 6 and 7. Add the Session handoff section and the `docs/HANDOFF.md` Docs entry to `AGENTS.md`. Change `CLAUDE.md` references to `AGENTS.md` in `README.md` and the PLAN.md Definition of done. Do not create `docs/HANDOFF.md` unless the developer asks.
    - **1.2.0 to 1.3.0:** Propose acceptance-criterion IDs in `SPEC.md`, with test cases grouped by feature or milestone, then create `docs/RELEASE-CHECKLIST.md` with one check per ID. Add its Docs entry and the release-check Never rule to `AGENTS.md`, and the two new Definition of done lines to `PLAN.md`. If the project has end users, offer to create `docs/USER-GUIDE.md`.
    - **1.3.0 to 1.4.0:** Add the `docs/archive/` Docs entry and the two new Never rules to `AGENTS.md`. If `docs/archive/` exists, propose a `docs/archive/INDEX.md` with one row per file; ask the developer for any status you cannot tell. Do not move or rename files already in the archive. If current docs still have superseded copies beside them (such as `SPEC-v2.md`), run steps 3 to 7 for them.
    - **1.4.0 to 1.5.0:** In `AGENTS.md`, replace the template sections with their 1.5.0 versions (the Making changes and Testing sections are new, and most others are shorter), and keep the project's own lines, such as Conventions and the Skills list. Keep only the Docs entries for files that exist, plus the always-listed ones, and add the "Not used" line. Add the Testing lines, including How to run tests and the results file, to `ARCHITECTURE.md`, and build its test map from the existing tests as in step 6. Update the PLAN.md Definition of done and the release steps in `RELEASE-CHECKLIST.md`. Add `test-results/` to `.gitignore` if the project uses version control. Change Jira headers to `> Jira: PROJ-# (name of project)`. Add the compaction note to the Claude-only notes in `CLAUDE.md`. Propose cuts to `AGENTS.md` under rule 8, and wait for approval. Offer the SPEC interview, and propose the new SPEC template guidance as SPEC changes.
    - **1.5.0 to 1.5.1:** In `AGENTS.md`, add the 2 new Making changes lines and the new Never rule. Extend the evidence line in the PLAN.md Definition of done with the two docs lines.

- **1.5.1 to 2.0.0:** The standard is now KEEL. New headings and template lines are structure: add them directly once the developer approves the plan, with "(Not documented yet.)" as content. Only changes to accepted SPEC or DESIGN text go through PROPOSED blocks. Headers keep their `> Jira:` form; the standard now accepts any tracker name.
    1. Root: during this version the root may hold `AI-PROJECT-DOCS-STANDARD.md` beside or instead of `KEEL.md`; follow `KEEL.md`. Set the stamp to `Aligned to KEEL 2.0.0 (https://github.com/DominusFL/keel)`. An existing stamp `Aligned to KEEL v2.0.0` is the same version in its earlier form: update it to the new form, and the `KEEL.md` line after it, the next time you edit `AGENTS.md`.
    2. `AGENTS.md`: replace every template section with its 2.0.0 version (all changed; Reporting and the precedence line are new; the cap is about 120 lines), keep the project's own lines, and add the Conventions lines for ask-before-touching paths and area owners. Propose cuts under rule 8 and wait for approval.
    3. `ARCHITECTURE.md`: convert the test map to the area map (add Code, Entry points, ACs, and Confidence; mark rows Inferred); add `Verified: (not yet verified)` under the header; add the Operations section with Known traps, the data lifecycle line, and the Testing lines for the docs check, the AC citation, and the results files; write the docs check script and add its row.
    4. `SPEC.md`: add `Verified: (not yet verified)`, the Glossary and Constraints headings, and the Area name on each feature group. Convert each proposed SPEC or DESIGN change waiting in the old `HANDOFF.md` Open questions into a PROPOSED block.
    5. `PLAN.md`: add the Definition of ready, replace the Definition of done, and convert Current / Next to the work board.
    6. `RELEASE-CHECKLIST.md`: replace the Release steps and the Result values.
    7. `README.md`: add Owners and support.
    8. `CLAUDE.md`: extend the compaction note.
    9. `HANDOFF.md`: run the close routine on it: route each item, move lasting gotchas to Known traps or `RESEARCH.md`, and rewrite it with the dated sections. If more than 1 developer works on the project, create `docs/handoff/<developer>.md` per developer and retire `docs/HANDOFF.md`.
    10. Offer the as-built review; it sets the Verified dates.
9. **Report** what moved where, what was inferred, which Superseded content was not migrated and what replaced it, what was left unmapped, and any conflicts. Also list, with file names, each superseded approach that the code still appears to use (possible dead code or an unfinished change). Run the Initialization checklist and mark any item that does not pass. Then offer the as-built review.

Changelog

Add a new entry at the top for each update, with its version and date, and update the version stamp in the `AGENTS.md` template to match.

- **2.0.0 (October 31, 2026):** Renamed to KEEL; any tracker, not only Jira. Handoff: a close routine with a printed routing report, a 40-line cap, dated bullets in 5 typed sections, and per-developer files for teams. Drift: SPEC and DESIGN proposals are PROPOSED blocks in the file, the test map became the area map, and a required docs check, a drift report, and Verified dates were added. Guardrails: risk classes, a ready card (rule 10), evidence in a fixed task summary (rule 11), a bug repro gate, no weakened tests, and a precedence line in `AGENTS.md`. Existing code: a 6-step as-built review. Teams: a work board with a claim rule in `PLAN.md`. New content: Operations, Known traps, Glossary, Constraints, Owners and support, and dependency review. `AGENTS.md` is capped at about 120 lines. The KEEL skill: 6 skills (`keel` , `keel-init` , `keel-upgrade` , `keel-check` , `keel-as-built` , `keel-interview`) generated from this file by `build/build.py` and installed once at user level, through the Claude Code plugin marketplace in this repository, `npx skills add` , or `install.sh` , for Claude Code, Codex, Cursor, and GitHub Copilot; a Switch to the skill path; and the neutral stamp `Aligned to KEEL 2.0.0 (https://github.com/DominusFL/keel)` , the same for the file and the skill.
- **1.5.1 (October 19, 2026):** Doc updates are no longer skipped: the AI searches `docs/` for what it changed, proposes SPEC and DESIGN edits in the same task, never defers doc updates to `HANDOFF.md` , and lists docs updated and checked in each task summary.

- **1.5.0 (October 19, 2026):** Added progressive testing (rule 9): targeted tests by default, a test map in `ARCHITECTURE.md`, scripted test runs, and a full regression only at releases or with developer approval. `RELEASE-CHECKLIST.md` is now required. Added the recommended `CHANGELOG.md`, a "Not used" line for removed files, an upgrade-only path, a fresh-context review before releases, an optional SPEC interview, plan-first and stop-after-2-failures rules, evidence in task summaries, a single setup prompt with automatic detection, and an optional as-built review. Shortened the introduction and the `AGENTS.md` template, clarified `DESIGN-GUIDE.html`, and added a compaction note to `CLAUDE.md`.
- **1.4.0 (October 14, 2026):** Migration now finds superseded documentation and migrates only current content. Added the Resolve currency step and currency table, decision notes and research entries for choices that changed, a report of superseded approaches still in the code, path-preserving archiving with `docs/archive/INDEX.md`, and an upgrade path. Added rules against new versions of a doc beside the old one and against reading the archive as current. The `AGENTS.md` Docs list now follows the table order.
- **1.3.0 (October 12, 2026):** Added the recommended `RELEASE-CHECKLIST.md` release gate, the optional `USER-GUIDE.md` (written in Simplified Technical English) and `images/` folder, and acceptance-criterion IDs with grouped test cases in SPEC. Also added optional Copilot-only notes and tightened the wording.
- **1.2.0 (October 11, 2026):** Moved the AI working rules to the shared `AGENTS.md`, which `CLAUDE.md` imports and `.github/copilot-instructions.md` points to. Added the recommended `HANDOFF.md` session handoff and upgrade steps from 1.1.0.
- **1.1.0 (October 10, 2026):** The document can now be used directly as a prompt. Added workflow, authority, and initialization rules; required, recommended, and optional statuses; optional `RESEARCH.md`; migration steps; and an initialization checklist.
- **1.0.0 (October 9, 2026):** Initial version.